

Conformance, Cost, and Replication in Constrained LLM Coding Agents

Alessio Rocchi¹

¹AIGEN Solutions

Version 1.4.3 — May 12, 2026

Abstract

Procedural protocols deliver +0.59 to +0.78 expected-utility gain over free prompting at log-token ratios 0.70–3.28 on 8 cells $\times$ 4 frontier models. However, 33–68% of procedurally-compliant runs still fail held-out semantic tests, exposing a conformance-without-semantics regime invisible to compliance-only metrics. We present a measurement framework that projects agent trajectories onto a behavior signature $B = \varphi(\tau)$ over conformance, target-correctness, robustness, and token cost, with a decisional cost-aware metric $\mathcal{V}$ admitting a closed-form break-even $\lambda_{\text{breakeven}}$ below which the protocol pays.

E1 validates empirical FOSD on the held-out utility grid ($n = 120$, all 80 thresholds; sample-level, not population). **L1** (asymmetric-quorum joint-miss bound, inclusion–exclusion arithmetic) is validated within the finance contract domain ($\bar{p} : 0.62 \rightarrow 0.29$, joint-miss 22.91% $\rightarrow$ 12.28%, paired McNemar $p = 0.0013$ on $n = 200$ bugs $\times$ 1,400 reviews); Phase-5 falsifies cross-domain transfer (`bankcheck.py`, $p \geq 0.5$ in all 5 contrasts). **C-T3** converges in simulation but the LLM-grounded counterpart shows no improvement under two pre-registered injection formats (abstract-name and concrete-example, $p \in [0.95, 0.97]$ against H_1 at $n = 92$), a simulation-to-LLM gap reported as absence of detected improvement, not evidence of absence. **C1** (contract-aligned specificity) replicates across 3 reviewer families on 4 phases; Phase-11 provenance-label falsification ($n = 60$, both pre-registered hypotheses fail at $p \in [0.13, 0.34]$) rules out anti-stdlib carelessness as the dominant mechanism, leaving ≥ 40 pp of the cross-sample detection gap to code-intrinsic factors. A naturalistic Phase-12 replication on CPython `csv.py` bugs harvested from the issue tracker is underpowered at the harvested sample size ($n = 8$, below the pre-registered target of 25–30); 0/3 families reach the Bonferroni-corrected primary threshold (Codex $p = 0.029$, Gemini $p = 0.031$, Opus $p = 0.50$) and Codex output is contaminated by 100% CPython issue-number leakage in the out-of-band probe—naturalistic C1 transfer at low n with the leakage caveat is therefore *reported but not claimed*.

The contribution is methodological discipline—explicit separation of *proved*, *measured*, *conjectured*, and *empirically falsified*—not a new prompting technique. Every numerical claim recomputes from raw artefacts in the companion repository.

Keywords: LLM code generation; procedural constraints; stochastic dominance; asymmetric review; error decorrelation; predictability–cost trade-off; AST-level mutation testing; reviewer-family variation.

Status: v1 manuscript with frozen pre-registrations preceding every reviewer call, computational validation of every numerical claim, and an iterative internal-review trail summarized in Section 8.6. Reproduction artefacts in the companion repository `agentic-sdlc/experiments/`.

Contents

1 Introduction

4

1.1	Motivation: what does a constrained agent actually buy?	4
1.2	Contributions	4
1.3	Scope and what we do not claim	5
1.4	Reproducibility	5
1.5	Paper structure	5
2	Related Work	6
2.1	LLM coding agents and procedural protocols	6
2.2	Mutation testing and AST-level operators	6
2.3	Asymmetric multi-agent review and error decorrelation	6
2.4	Stochastic dominance in software measurement	7
2.5	Specification-first and contract-driven review	7
2.6	Where this paper sits	7
3	Notation and Load-Bearing Primitives	7
3.1	Agent as a stochastic transducer	7
3.2	Protocol	8
3.3	Held-out condition (information constraint)	8
3.4	Behavior signature, utility, and the conformance/semantics split	8
3.5	Distributions induced by the protocol	8
3.6	Notation summary (load-bearing primitives)	9
3.7	Pointer to the CMDP extension technical report	9
4	The Decisional Metric $\mathcal{V}$	9
4.1	Break-even λ	9
4.2	Empirical first-order stochastic dominance (eFOSD)	10
4.3	Sample-level inequality and its empirical certification	10
4.4	Empirical Claim E1 (Expected-Utility Dominance + eFOSD)	10
4.5	Reading the metric: three regimes	11
4.6	What $\mathcal{V}$ does <i>not</i> capture	11
5	Asymmetric Verification and the Joint-Miss Bound	11
5.1	Reviewer ensemble	11
5.2	The 2-of-3 quorum decision rule	12
5.3	Joint-miss bound (Lemma L1)	12
5.4	Mechanism: prompt asymmetry reduces ρ_{ij}	12
5.5	Model diversity vs prompt asymmetry: a 2×2 factorial	13
5.6	Domain restriction: Phase 5 falsification	13
6	Learning Loop: Conjecture C-T3 and the Simulation-to-LLM Gap	13
6.1	Pattern-extraction loop	13
6.2	Conjecture C-T3 (Learning-Loop Convergence)	14
6.3	LLM-grounded counterpart	14
6.4	Phase-5 falsification: abstract-name injection	15
6.5	Phase-5 E3v2: concrete-example injection	15
6.6	Simulation-to-LLM gap	15
7	Experiments	15
7.1	Phase 1–3: pilot ($n = 120$ trajectories, 4 frontier models)	16
7.2	Phase 4 ($n = 120$): E1 validation, expected utility + eFOSD	16
7.3	Phase 5 ($n = 27 + n = 92$): L1 cross-domain falsification and C-T3 LLM-grounded falsification	16

7.4	Phase 6: Cold reviewer recovery via auto-extracted contracts	16
7.5	Phase 7: Second domain (JSON parser)	16
7.6	Phase 8: Three hand-written domains (multi-domain pilot)	16
7.7	Phase 9: Real Python stdlib codebases	17
7.8	Phase 10: Third-party Python libraries (3 domains $\times$ 3 families $\times$ 2 conditions $\times$ 30 bugs)	17
7.9	Phase 11: Provenance-label experiment	18
7.10	Phase 12: Naturalistic bug benchmark on csv.py ($n = 8$)	19
7.11	Empirical summary	20
8	Threats to Validity and Discussion	20
8.1	Construct validity	21
8.2	Internal validity: ten failure modes (operationally defined)	21
8.3	External validity	22
8.4	Conclusion validity	22
8.5	What this paper has and has not established	23
8.6	Internal-review trail and frozen-pre-registration discipline	24
9	Conclusion	24
9.1	Phase-12 desiderata	25
9.2	Final stance	26
A	Statistical Methods Appendix	26
A.1	Paired McNemar exact one-sided test [17]	26
A.2	Wilson score 95% confidence interval [24]	27
A.3	Fisher’s combined p -value [8]	27
A.4	DKW concentration for eFOSD	27
A.5	Mixed-effects logistic model (primary) and comparators	27
A.6	Parse-failure policies	29
A.7	Detection-criterion specification	29
A.8	Pre-registration discipline	29
A.9	C1 per-cell three-way comparison (full table)	30

1 Introduction

1.1 Motivation: what does a constrained agent actually buy?

LLM-based coding agents have moved from one-shot autocomplete to multi-step autonomous workflows: spec-first design, test-driven implementation, multi-role code review, persistent decision traces. Each procedural constraint exchanges a degree of model freedom for a presumed gain in output predictability or downstream correctness. The exchange is not free: every additional verifier call, every spec-then-test cycle, every persisted artefact adds tokens, latency, and engineering complexity. The empirical question is sharp:

Given a coding task, a model, and an inference budget, does adding procedural structure measurably improve the distribution of outcomes, and at what cost?

This paper proposes a measurement methodology—not a general theory—that answers this question on operationally well-defined components: target-correctness, robustness, conformance, and token cost. We deliberately separate *what we can prove formally*, *what we can measure*, *what we conjecture*, and *what we can empirically falsify*. Every numerical claim is reproducible from artefacts in the companion repository.

1.2 Contributions

We make four contributions, three positive and one negative.

1. A **minimal formal model** (Section 3) of a procedurally-constrained LLM coding agent at the level of *load-bearing primitives only*: stochastic transducer, verifier kernels (stochastic, accommodating LLM-based reviewers), held-out information constraint $Y_S \perp h_t$ (Equation (3)), behavior signature $B = \varphi(\tau)$, conformance/semantics predicates $\text{Pred}_C, \text{Pred}_S$ (Equations (6) and (7)), and the decisional cost-aware metric $\mathcal{V}$ with explicit notational disambiguation from V . The full CMDP extension (state augmentation, regime modes, escalation map, mode-conditioned action sets) is published as a standalone technical report (Section 3.7; docs/paper/v1.4/cmdp-extension-techreport.md) for future work requiring policy-space dynamics; it is *not used* by any empirical result of this paper.
2. A **decisional metric** $\mathcal{V}$ (Section 4) and an **eFOSD** validation protocol (Definition 2) that admit a closed-form break-even cost weight $\lambda_{\text{breakeven}}$ below which the protocol pays. Validated on $n = 120$ trajectories across 2 tasks $\times$ 4 frontier models with explicit Wilson confidence intervals and parse-failure sensitivity analyses.
3. Two **empirical claims with explicit domain restrictions**:
 - **L1 (Asymmetric-Quorum Joint-Miss Bound, finance-restricted)**: validated in 1,400 LLM reviews ($n = 200$ injected bugs), error decorrelation $\bar{\rho}: 0.62 \rightarrow 0.29$, joint-miss $22.91\% \rightarrow 12.28\%$, paired McNemar $p = 0.0013$. *Falsified across domains* in Phase 5 (bankcheck.py, $p \geq 0.5$ in all 5 contrasts); the cold contract-first reviewer collapses to 11% when contracts are misaligned with the domain.
 - **C1 (Cold reviewer transfer)**: contract-aligned specificity replicates across 3 reviewer families on a 3-library third-party sample (Section 7.8); library-class is a candidate but unconfirmed moderator (provenance-label experiment in Section 7.9 tests this directly). A naturalistic replication on CPython csv.py bugs (Section 7.10, $n = 8$ after strict inclusion) is underpowered at the harvested sample size: 0/3 families reach the Bonferroni-corrected primary threshold and the Codex result is contaminated by 100% training-set leakage of CPython issue numbers. Naturalistic C1 transfer at low n with the leakage caveat is *reported but not claimed*; a leakage-controlled benchmark is a Phase-13 desideratum.

4. A **negative result with theoretical and empirical components: C-T3 (Learning-Loop Convergence)** *converges almost surely* in simulation under conservative trigger, bounded injection, and weak suppression, but the LLM-grounded counterpart *shows no improvement* under two pre-registered injection formats (abstract-name and concrete-example, both at $p \in [0.95, 0.97]$ against H_1 “injected $>$ baseline”). This documents a *simulation-to-LLM gap*: convergence properties of pattern-extraction loops do not transfer to measurable detection improvements at realistic n .

A consolidated **ten-mode failure taxonomy** (Section 8) classifies the regimes under which the framework breaks: uniform-failure collapse (the regime that broke our pilot’s Shannon-entropy formulation), oracle leakage, symmetric-reviewer anchoring, model-strength ceiling (Claude-vs-Codex confounder), conformance-without-semantics, off-by-one blind spots, out-of-domain prompt collapse, simulation-to-LLM gap, sample-dependent reviewer-choice sensitivity, and descriptive injection-anchoring effects.

1.3 Scope and what we do not claim

We are explicit about scope:

- We do not claim that constrained protocols are universally beneficial; the break-even $\lambda_{\text{breakeven}}$ is task- and model-specific.
- We do not claim L1 generalizes outside the finance contract domain; Phase 5 shows transfer requires re-tooled contracts.
- We do not claim C1 establishes a general property of cold-reviewer prompts; the absolute-detection rate varies across reviewer families and library classes in ways that are not yet causally attributed.
- We do not propose new theorems in stochastic dominance, mutation testing, or CMDP theory; we apply established tools as measurement instruments.

1.4 Reproducibility

Every numerical claim in this paper is recomputed from raw artefacts by a validation script in `docs/paper/validations/` (Phases 4–6) or in each phase’s experiment directory (Phases 7–11). Raw reviewer outputs, AST mutator code, pre-registration documents, and Codex peer-review reports are committed to the repository at the indicated paths. Reviewer prompts, contracts (auto-extracted and mismatched-bankcheck), and reference modules are bit-identical to those used at runtime.

1.5 Paper structure

Section 2 surveys related work on LLM agents, mutation testing, and stochastic dominance applied to code-generation systems. Section 3 fixes the load-bearing primitives (stochastic transducer, verifier kernels, held-out condition, behavior signature); the full CMDP extension is deferred to a standalone technical report (Section 3.7). Section 4 introduces $\mathcal{V}$, defines empirical FOSD, and states Empirical Claim E1. Section 5 states the joint-miss bound (Lemma L1) and reports its empirical mechanism within the finance domain. Section 6 formalizes C-T3 and reports the simulation-to-LLM gap. Section 7 reports the experimental phases 1–11 with pre-registered metrics and computational validation. Section 8 catalogues the ten failure modes and discusses external validity, including the cross-phase model-version confounder. Section 9 closes with explicit Phase-12 desiderata.

2 Related Work

2.1 LLM coding agents and procedural protocols

The transition from one-shot completion to multi-step agentic workflows has produced a body of work on *prompting protocols* (chain-of-thought [23], ReAct [28], Reflexion [22], Self-Refine [15]), *multi-agent debate and review* [26, 5], and *verifier-guided generation* [27]. Most existing work measures *task success rate* on benchmark suites (HumanEval [3], MBPP [2], SWE-bench [11], BigCodeBench [30], EvalPlus [14]) and reports point estimates. Our framework differs in three respects: (i) the behavior signature $B = \varphi(\tau)$ separates conformance, target-correctness, robustness, and token cost as orthogonal axes; (ii) we use *empirical first-order stochastic dominance* (Definition 2) following the classical FOSD literature [9, 13] rather than mean comparison, which is more conservative and finer-grained; (iii) we explicitly formalize held-out oracles (Equation (3)) to avoid the data-contamination problem that compromises many existing benchmarks [16, 20, 29].

2.2 Mutation testing and AST-level operators

The AST-level mutation operators we use in Phases 9–11 (Definition 1 below) follow the standard mutation-testing literature [4, 19, 1, 10], and the broader controlled-bug benchmark tradition exemplified by Defects4J [12].

Definition 1 (AST mutation operators (PRIORITY 1)). Let T be the AST of a Python module. The three mutation-operator families are:

AOR (Arithmetic Operator Replacement) for each `BinOp` node with $\text{op} \in \{+, -, *, /, //, \%, \}$,
swap $+$ $\leftrightarrow$ $-$, $*$ $\leftrightarrow$ $/$, $//$ $\leftrightarrow$ $\%$.

ROR (Relational Operator Replacement) for each `Compare` node with $\text{op} \in \{==, \neq, <, \leq, >, \geq\}$, swap $==$ $\leftrightarrow$ $\neq$, $<$ $\leftrightarrow$ $\leq$, $>$ $\leftrightarrow$ $\geq$.

BOR (Boolean Operator Replacement) for each `BoolOp` node, swap `and` $\leftrightarrow$ `or`.

A mutation is *valid* iff the mutated source AST is parseable, and (under Phase-9 strict validation) importable as a standalone module; under Phase-10 loose validation (third-party with package imports), parseable and compilable.

Most prior LLM-code-review work uses *text-substitution* bugs (find/replace strings in source) which we found in Phase 8 to have only 44% yield (16/36 attempted injections compiled). The AST-mutator approach achieves 100% yield by validating each mutant at the AST/compile level. We do not claim novelty for the mutators themselves; the contribution is applying them as a frozen, pre-registered bug-injection benchmark for cold-reviewer evaluation.

2.3 Asymmetric multi-agent review and error decorrelation

The asymmetric-quorum joint-miss bound (Equation (17)) builds on classical work on error correlation in ensembles. The novelty is the operational reduction: we show empirically that prompt asymmetry (*warm + skeptic + cold contract-first*) reduces pairwise reviewer correlations from $\bar{\rho} = 0.62$ to 0.29 on a fixed bug suite, and that model diversity (4 frontier models with one cold reviewer per role) is required to obtain the second-largest correlation drop. The Phase-5 falsification on `bankcheck.py` demonstrates that the mechanism is *contract-domain-bound*: when the cold reviewer’s contracts are misaligned with the test domain, decorrelation collapses.

2.4 Stochastic dominance in software measurement

First-order stochastic dominance has been used in software measurement primarily for reliability comparisons. We adapt it to LLM-agent measurement by defining eFOSD on a held-out utility threshold grid (Definition 2) and verifying it on 10 thresholds per cell across 8 cells (Phase 4). This is conservative: a single threshold violation falsifies the empirical claim, but no such violation occurred in our data.

2.5 Specification-first and contract-driven review

Contract-driven code review has roots in Design-by-Contract [18]. Our cold contract-first reviewer is a re-instantiation in the LLM setting, with one twist: the contract document is *auto-extracted* from the reference implementation by an LLM call (Phases 6–11), removing the manual-contract bottleneck that prevented prior empirical replication. The Phase-9 / Phase-10 results suggest the technique is most effective when (a) contracts are domain-aligned and (b) the codebase is unfamiliar to the reviewer (Phase 10 third-party libs vs Phase 9 stdlib).

2.6 Where this paper sits

This paper is positioned as a *measurement framework* rather than a benchmark or a new prompting technique. The contributions are:

- the formal model and decisional metric (Sections 3 and 4);
- the empirically-validated L1 bound *within its declared domain* (Section 5);
- the C-T3 conjecture and the absence of detected LLM-grounded improvement under two pre-registered injection formats at $n = 92$ paired (Section 6);
- the four-phase replication of C1 contract-aligned specificity (Sections 7.4 to 7.8) with explicit reviewer-family $\times$ library-class factorial design;
- the Phase-11 provenance-label experiment (Section 7.9) that disentangles label vs code-intrinsic causal contributions;
- the ten-mode failure taxonomy (Section 8).

3 Notation and Load-Bearing Primitives

This section defines the minimal set of primitives used in Sections 4 to 6. The full CMDP construction (state augmentation with regime modes, escalation map, mode-conditioned action sets) is published as a standalone technical report (Section 3.7); it is an extension that this paper does not require for any empirical result.

3.1 Agent as a stochastic transducer

Let $x \in \mathcal{X}$ be the task specification and $\mathcal{Y}$ the model response space (token strings). The LLM is a parametric stochastic kernel

$$M_\theta(y_t \mid x, h_t), \quad y_t \in \mathcal{Y}, \quad (1)$$

where h_t is the observable history $((a_1, o_1), \dots, (a_{t-1}, o_{t-1}))$. A deterministic parser maps responses to actions: $\pi_{\text{act}} : \mathcal{Y} \rightarrow \mathcal{A} \cup \{\perp\}$. We treat M_θ as a black box.

A trajectory is the sequence $\tau = ((a_1, o_1), \dots, (a_T, o_T))$. The *final artefact bag* $\alpha(\tau)$ is the multiset of files, test results, and decision-trace records produced during the run.

3.2 Protocol

A protocol is a tuple $\mathcal{P} = (\mathcal{A}, V)$ at the level of generality we need, where $\mathcal{A}$ is the action set and $V = (V_1, \dots, V_K)$ is a finite list of *verifier kernels*. Each V_k is a stochastic kernel

$$V_k(v_k = \cdot \mid \tau_{1:t-1}, a_t, \text{env}_t), \quad v_k \in \{\text{accept}, \text{reject}, \text{escalate}\}. \quad (2)$$

The kernel form (i.e., V_k is a random variable, not a function) accommodates noisy verifiers (LLM-based reviewers, flaky test runners). **This is load-bearing for Equation (17)**: the miss-indicators M_i of Section 5 are well-defined as binary random variables only because V_k is a kernel.

Verifiers split into protocol-internal $V_C \subset V$ and held-out oracles $V_S \subset V$. *Free prompting* $\mathcal{P}_0$ is the degenerate protocol with $\mathcal{A} = \mathcal{A}_{\text{free}}$ and $V = \emptyset$.

3.3 Held-out condition (information constraint)

Let $Y_S(\tau)$ denote the random vector of V_S verdicts on τ . The held-out condition is

$$Y_S(\tau) \perp h_t \quad \text{for all } t \leq T, \quad (3)$$

i.e., the held-out verdicts are conditionally independent of the agent’s observable history given the task x . Operationally: no V_S verdict appears in any o_t and no proxy of V_S (test outputs, traces, hints) appears in o_t either. **This is load-bearing** for the Pred_C vs Pred_S distinction (Equations (6) and (7)) and for ruling out oracle leakage as a failure mode (Section 8).

3.4 Behavior signature, utility, and the conformance/semantics split

The behavior signature is the deterministic projection

$$B = \varphi(\tau) = (c_{\text{form}}(\tau), c_{\text{corr}}(\tau), c_{\text{rob}}(\tau), c_{\text{tok}}(\tau)), \quad (4)$$

with $c_{\text{form}}(\tau) \in [0, 1]$ the fraction of V_C verdicts accepted along τ , $c_{\text{corr}}(\tau) \in [0, 1]$ the fraction of V_S held-out tests passed by $\alpha(\tau)$, $c_{\text{rob}}(\tau) \in [0, 1]$ the mutation-killing rate of agent-produced tests against a fixed mutant set, and $c_{\text{tok}}(\tau) \in \mathbb{N}$ the total token cost.

The target utility is

$$u(B) = \alpha c_{\text{corr}} + \beta c_{\text{rob}} + \gamma c_{\text{form}}, \quad \alpha + \beta + \gamma = 1, \quad \alpha + \beta > \gamma. \quad (5)$$

The choice $\alpha + \beta > \gamma$ ensures the utility is dominated by held-out components, avoiding the conformance-without-semantics trap.

The *conformance / semantics split* is operationalized by

$$\text{Pred}_C(\tau; \mathcal{P}) := \mathbb{1}[\text{all } V_C \text{ verdicts on } \tau \text{ are } \textit{accept}], \quad (6)$$

$$\text{Pred}_S(\tau; \mathcal{P}) := \mathbb{1}[\text{all } V_S \text{ verdicts on } \alpha(\tau) \text{ are } \textit{accept}]. \quad (7)$$

Two regimes are pathological: *conformance without semantics* ($\mathbb{E}_{\mathcal{P}}[\text{Pred}_C] \gg \mathbb{E}_{\mathcal{P}}[\text{Pred}_S]$, quantified at 33–68% in our $\mathcal{P}$ -condition data, see Section 7.2) and *semantics without conformance* (rare, $\leq 7\%$).

3.5 Distributions induced by the protocol

A protocol $\mathcal{P}$, a fixed model M_θ , and a fixed task x induce a distribution over trajectories and hence over signatures:

$$P_B^{\mathcal{P}, x} := \text{law of } \varphi(\tau) \text{ under } M_\theta, \pi_{\text{act}}, V, \text{ given } x. \quad (8)$$

We compare $\mathcal{P}_0$ and $\mathcal{P}$ by comparing $P_B^{\mathcal{P}_0, x}$ and $P_B^{\mathcal{P}, x}$ through u , stochastic dominance, and $\mathcal{V}$.

Remark 1 (Notation: V vs $\mathcal{V}$). The symbol V denotes verifier kernels (Equation (2)). The decisional cost-aware metric defined in Equation (9) is denoted $\mathcal{V}$ (script V) to avoid notational collision.

3.6 Notation summary (load-bearing primitives)

Symbol	Meaning	Used in
$M_\theta(y_t x, h_t)$	LLM stochastic kernel	E1, L1, C-T3, C1
$\tau, \alpha(\tau)$	Trajectory; final artefact bag	all
V_k kernels; V_C, V_S	Verifier kernels; internal/held-out	L1 (kernel form is load-bearing)
$Y_S \perp h_t$	Held-out information constraint	failure mode #2
$B = \varphi(\tau)$	Behavior signature	all
$u(B), \alpha, \beta, \gamma$	Target utility, $\alpha + \beta + \gamma = 1, \alpha + \beta > \gamma$	E1, $\mathcal{V}$
$\text{Pred}_C, \text{Pred}_S$	Conformance / semantics predicates	failure mode #5
$P_B^{\mathcal{P}, x}$	Behavior-signature distribution	E1
$\mathcal{V}$ (script V)	Decisional cost-aware metric	Equation (9)

Primitives used in the CMDP extension technical report (Section 3.7) but *not* in the main paper: the full CMDP state σ_t , the artefact-bag accumulator α_t , the regime-mode set $\mathcal{M}$, the escalation map Ω , and the mode-conditioned action sets $\mathcal{A}_m$.

3.7 Pointer to the CMDP extension technical report

The full CMDP extension (state augmentation with regime modes, escalation map, mode-conditioned action sets) is published as a standalone technical report (see `docs/paper/v1.4/cmdp-extension-te` in the companion repository). It is offered as an interface for future work that requires policy-space dynamics; no empirical result of this paper depends on it.

4 The Decisional Metric $\mathcal{V}$

The decisional cost-aware metric weighs the expected-utility gain of a protocol $\mathcal{P}$ over free prompting $\mathcal{P}_0$ against the log token-cost ratio:

$$\boxed{\mathcal{V}(\mathcal{P}; \mathcal{P}_0; \lambda) = \Delta\mathbb{E}[u] - \lambda \log \frac{K_{\mathcal{P}}}{K_{\mathcal{P}_0}}} \quad (9)$$

where $K_{\mathcal{P}} := \mathbb{E}_{P_B^{\mathcal{P}, x}}[c_{\text{tok}}]$ is the expected token cost under protocol $\mathcal{P}$ on task x , and $\Delta\mathbb{E}[u] := \mathbb{E}_{P_B^{\mathcal{P}, x}}[u(B)] - \mathbb{E}_{P_B^{\mathcal{P}_0, x}}[u(B)]$.

The cost weight $\lambda \in [0, \infty)$ encodes how much the decision-maker is willing to pay (in log-token units) for a unit of expected utility gain. The structure is Lagrangian: $\mathcal{V} > 0$ certifies that the protocol pays.

4.1 Break-even λ

The break-even cost weight is the unique λ at which $\mathcal{V} = 0$:

$$\lambda_{\text{breakeven}}(\mathcal{P}; \mathcal{P}_0; x) = \frac{\Delta\mathbb{E}[u]}{\log(K_{\mathcal{P}}/K_{\mathcal{P}_0})} \quad \text{whenever } K_{\mathcal{P}} > K_{\mathcal{P}_0}. \quad (10)$$

Below $\lambda_{\text{breakeven}}$ the protocol pays; above it free prompting pays. This is a one-dimensional summary of the cost-utility trade-off, computable from any matched pair of trajectory samples.

We define the cost-efficiency ratio

$$G(\mathcal{P}; \mathcal{P}_0; x) = \lambda_{\text{breakeven}}(\mathcal{P}; \mathcal{P}_0; x) \quad (11)$$

to keep λ reserved for the decision-maker’s actual weight.

4.2 Empirical first-order stochastic dominance (eFOSD)

Mean comparison is one-dimensional and can be misleading. We use a stronger sample-level condition:

Definition 2 (eFOSD). Let $P_B^{\mathcal{P},x,(n)}$, $P_B^{\mathcal{P}_0,x,(n)}$ be empirical distributions of $u(B)$ under $\mathcal{P}$ and $\mathcal{P}_0$, each with n Monte-Carlo replicas. Let $\mathcal{T} = \{t_1, \dots, t_K\} \subset [0, 1]$ be a held-out utility threshold grid. We say $P_B^{\mathcal{P},x,(n)}$ *empirically first-order stochastically dominates* $P_B^{\mathcal{P}_0,x,(n)}$ on $\mathcal{T}$ iff

$$\mathbb{P}_{P_B^{\mathcal{P},x,(n)}}[u(B) \geq t] \geq \mathbb{P}_{P_B^{\mathcal{P}_0,x,(n)}}[u(B) \geq t] \quad \forall t \in \mathcal{T}. \quad (12)$$

We write $P_B^{\mathcal{P},x} \succeq_{\text{eFOSD},\mathcal{T}} P_B^{\mathcal{P}_0,x}$.

Remark 2. eFOSD is strictly stronger than mean comparison: if it holds on a dense enough $\mathcal{T}$, expected utility under $\mathcal{P}$ dominates under $\mathcal{P}_0$ for any non-decreasing utility transformation. It is strictly weaker than population FOSD: we do not assert dominance on thresholds *outside* $\mathcal{T}$. A single threshold violation falsifies the empirical claim, so the definition is self-falsifiable.

In our experiments, $\mathcal{T}$ is a 10-point grid uniformly spaced in $[0.1, 1.0]$, applied to 8 cells (2 tasks $\times$ 4 models). eFOSD is verified on 80/80 thresholds in Phase 4 (Section 7.2).

4.3 Sample-level inequality and its empirical certification

Lemma 1 (Sample-level eFOSD certification). *Given n matched-task replicas $(\tau_{\mathcal{P}}^{(i)}, \tau_{\mathcal{P}_0}^{(i)})_{i=1}^n$, the empirical exceedance probability*

$$\hat{F}_{\mathcal{P}}(t) := \frac{1}{n} \sum_{i=1}^n \mathbb{1}[u(\varphi(\tau_{\mathcal{P}}^{(i)})) \geq t] \quad (13)$$

admits the Dvoretzky–Kiefer–Wolfowitz inequality $\mathbb{P}[\sup_t |\hat{F}_{\mathcal{P}}(t) - F_{\mathcal{P}}(t)| > \epsilon] \leq 2e^{-2n\epsilon^2}$, so the n -sample eFOSD condition certifies the population condition with confidence $1 - 4Ke^{-2n\epsilon^2}$ on a grid of size K at slack ϵ , by a union bound over the K thresholds.

We use this only as a sanity check: we report the empirical condition on the tested grid without claiming the population statement.

4.4 Empirical Claim E1 (Expected-Utility Dominance + eFOSD)

We do not state E1 as a proposition. The condition for the dominance to hold—that the protocol-allowed action sequences include at least one producing an artefact passing held-out tests—is nearly tautological for $\Delta\mathbb{E}[u] > 0$ on a task the protocol can in fact solve. The substantive content of E1 is the *empirical observation* on $n = 120$ trajectories:

Empirical Claim E1. On $n = 120$ matched-task trajectories across 2 tasks and 4 frontier models (Codex gpt-5.5, Claude Sonnet 4.6, Claude Opus 4.7, Gemini 2.5 Flash), with the protocol vs free-prompting conditions defined in Section 3:

1. $\Delta\mathbb{E}[u] \in [+0.59, +0.78]$ across all 8 cells.
2. $P_B^{\mathcal{P},x,(n)} \succeq_{\text{eFOSD},\mathcal{T}} P_B^{\mathcal{P}_0,x,(n)}$ on a 10-point grid $\mathcal{T}$ uniformly spaced in $[0.1, 1.0]$, verified on 80/80 thresholds.
3. $\log K_{\mathcal{P}}/K_{\mathcal{P}_0} \in [0.70, 3.28]$ (token ratios $[2.0\times, 26.6\times]$); $G \in [0.21, 1.11]$.

Status. *Empirically measured* on the stated sample. We do not assert population FOSD; we do not assert generalization beyond the tested cells. eFOSD is self-falsifiable: a single threshold violation would falsify the empirical claim, and we report no such violation.

4.5 Reading the metric: three regimes

We distinguish three operational regimes for $(\Delta\mathbb{E}[u], K_{\mathcal{P}}/K_{\mathcal{P}_0})$:

1. **Dominant utility, modest cost** ($\Delta\mathbb{E}[u] > 0$, $K_{\mathcal{P}}/K_{\mathcal{P}_0} \in [1, 5]$): protocol pays for $\lambda < G \in [\Delta\mathbb{E}[u]/\log 5, \infty)$. Typical regime in Phase 4.
2. **Dominant utility, heavy cost** ($\Delta\mathbb{E}[u] > 0$, $K_{\mathcal{P}}/K_{\mathcal{P}_0} > 10$): protocol pays only at low λ . G is small. Phase-4 high-cost cells fall here.
3. **No utility gain or token-cost regression** ($\Delta\mathbb{E}[u] \leq 0$): $\mathcal{V} < 0$ for any $\lambda > 0$. Protocol does not pay; this is what we observe under uniform-failure collapse (Section 8).

The Phase-4 empirical findings (Section 7.2) place all 8 tested cells in regimes 1–2 with $\Delta\mathbb{E}[u] \in [+0.59, +0.78]$ and $\log K_{\mathcal{P}}/K_{\mathcal{P}_0} \in [0.70, 3.28]$, giving $G \in [0.21, 1.11]$.

4.6 What $\mathcal{V}$ does *not* capture

- **Latency / wall-clock cost.** $\mathcal{V}$ is in log-token units; latency is a separate dimension.
- **Engineering complexity.** The setup cost of writing the protocol is amortized across runs and not captured.
- **Risk preferences beyond linear utility.** The risk-neutral $\mathbb{E}[u]$ is the simplest aggregation; risk-averse decision-makers may prefer CVaR or VaR formulations, which we do not present.
- **Distributional aspects of c_{tok} .** We use $K = \mathbb{E}[c_{\text{tok}}]$. Heavy tails in token cost may matter; we report empirical quantiles in Section 7.2 but do not integrate them into $\mathcal{V}$.

5 Asymmetric Verification and the Joint-Miss Bound

We formalize the multi-reviewer verifier subsystem and state the joint-miss bound that motivates asymmetric prompt design.

5.1 Reviewer ensemble

Fix an ensemble of R stochastic reviewers $V_1, \dots, V_R$ in the protocol-internal verifier set V_C . On a bug-injection task with ground-truth oracle (whether the bug exists), each reviewer either *catches* the bug (verdict reject) or *misses* it (verdict accept). Let $M_i \in \{0, 1\}$ be the miss-indicator of reviewer i on a single bug instance,

$$M_i = \mathbb{1}[V_i \text{ verdict} = \text{accept}]. \quad (14)$$

Reviewer marginal miss probabilities are $p_i := \mathbb{E}[M_i]$ and pairwise correlations are $\rho_{ij} := \text{Corr}(M_i, M_j)$. We assume the joint distribution of $(M_1, \dots, M_R)$ is well-defined over the bug-instance distribution; we do not assume independence.

5.2 The 2-of-3 quorum decision rule

Throughout we use the *2-of-3 quorum*: a bug is declared *caught* by the ensemble iff at least two of the three reviewers vote reject. The ensemble *misses* the bug iff at most one reviewer rejects, i.e., at least two reviewers miss:

$$Q_{2/3} := \mathbb{1}[M_1 + M_2 + M_3 \geq 2]. \quad (15)$$

The *joint-miss probability* is

$$q := \mathbb{E}[Q_{2/3}] = \mathbb{P}[M_1 + M_2 + M_3 \geq 2]. \quad (16)$$

5.3 Joint-miss bound (Lemma L1)

The bound below is mathematically routine—inclusion–exclusion plus the standard binary-indicator joint. We state it as a lemma, not a proposition, because the contribution of this section is the *empirical mechanism* that follows from monotonicity in the pairwise correlations (Section 5.4), not the inequality itself.

For any joint distribution of binary miss-indicators (M_1, M_2, M_3) with marginals p_i and pairwise correlations ρ_{ij} ,

$$q \leq \sum_{1 \leq i < j \leq 3} \left[p_i p_j + \rho_{ij} \sqrt{p_i(1-p_i)p_j(1-p_j)} \right], \quad \text{monotone-increasing in each } \rho_{ij}. \quad (17)$$

1

5.4 Mechanism: prompt asymmetry reduces ρ_{ij}

The right-hand side of Equation (17) is monotone in each pairwise correlation ρ_{ij} , so q is reduced by any intervention that reduces the ρ_{ij} . Symmetric prompts (three identical *warm* reviewers, or three identical *skeptic* reviewers) anchor on shared interpretation modes and exhibit high $\bar{\rho}$. Asymmetric prompts (*warm* + *skeptic* + *cold contract-first*) diversify failure modes:

- *warm* optimizes for stakeholder-business intent;
- *skeptic* optimizes for adversarial trust assumptions;
- *cold* reads only the formal contract, ignoring style and stakeholder concerns.

A bug missed by *warm* (because it appears business-acceptable) may be caught by *cold* (because it violates the formal contract), and vice versa.

Empirical validation (within the finance contract domain, Phase 4): on $n = 200$ injected bugs across 7 categories with 1,400 LLM reviews,

$$\bar{\rho}_{\text{symm-mono}} = 0.62 \quad \bar{\rho}_{\text{asymm-multi}} = 0.29 \quad \Delta\bar{\rho} = -0.33$$

and the joint-miss probability is reduced from $q_{\text{symm-mono}} = 22.91\%$ to $q_{\text{asymm-multi}} = 12.28\%$, paired McNemar $p = 0.0013$ (Bonferroni-significant under all three parse-failure policies tested).

¹Equation (17) is the standard joint-probability bound for binary indicators; the inclusion–exclusion proof is one line and is omitted.

5.5 Model diversity vs prompt asymmetry: a 2×2 factorial

The 2×2 factorial in Phase 4 decomposes the prompt-asymmetry effect from the model-diversity effect:

	Single model (Codex)	4 models with cold-per-role
Symmetric prompts	symm-mono $q = 22.91\%$	symm-multi $q = 18.74\%$
Asymmetric prompts	asymm-codex $q = 16.55\%$	asymm-multi $q = 12.28\%$

Both prompt asymmetry and model diversity contribute independently to $\bar{\rho}$ reduction; the combination achieves the lowest q .

5.6 Domain restriction: Phase 5 falsification

L1’s bound holds mathematically without domain restriction; *the empirical reduction mechanism is contract-domain-bound*. In Phase 5, the cold contract-first reviewer is given *finance contracts* but the bugs are injected in a non-finance code-pipeline target (`bankcheck.py`). The cold reviewer’s detection collapses from $\sim 90\%$ to 11% because the contract document is misaligned with the target domain, so the reviewer cannot derive the right expectations.

The pre-registered Phase-5 contrasts on `bankcheck.py` ($n = 27$ multi-line bugs across 10 categories) all return $p \geq 0.5$ on paired McNemar against the baseline. **We accept this as a falsification of cross-domain transfer for L1’s empirical mechanism**, while the mathematical inequality of Equation (17) continues to hold; only its empirical relevance (the magnitude of $\bar{\rho}$ reduction) is domain-bound.

Recovery (Phase 6+): auto-extracted aligned contracts from the target domain restore cold reviewer detection. Validated in 4+ phases across stdlib and third-party libraries (Sections 7.4 to 7.8).

6 Learning Loop: Conjecture C-T3 and the Simulation-to-LLM Gap

We formalize the conservative pattern-extraction loop and state its convergence conjecture, then report the LLM-grounded counter-evidence from Phase 5 and the E3v2 follow-up.

6.1 Pattern-extraction loop

Let $\mathcal{C}$ denote a finite set of *bug categories* (`off-by-one`, `logic`, `currency`, `atomicity`, `exception`, $\dots$). Each review cycle $k \in \mathbb{N}$ produces, for each category $c \in \mathcal{C}$:

- a count of bugs encountered $n_c^{(k)}$;
- a count of bugs *missed* (verdict accept when the ground truth is reject) $m_c^{(k)}$;
- the miss rate $r_c^{(k)} := m_c^{(k)} / \max(1, n_c^{(k)})$.

The loop maintains a top- K *injection set* $\mathcal{I}^{(k)} \subseteq \mathcal{C}$ of size $|\mathcal{I}^{(k)}| \leq M$. At each cycle, $\mathcal{I}^{(k+1)}$ is updated by:

1. **Trigger** (conservative, $\theta \geq 2$): a category c is eligible for injection only if $n_c^{(k)} \geq \theta$ (at least θ instances observed).
2. **Selection**: among triggered categories, the top- K by miss rate $r_c^{(k)}$ are kept.
3. **Bounded injection** ($M < \infty$): $|\mathcal{I}^{(k+1)}| \leq M$ to prevent unbounded prompt growth.

4. **Weak suppression** ($\varepsilon \in (0, 1)$): a category’s injection effect on subsequent miss rate is bounded; concretely, miss rate after injection satisfies $r_c^{(k+1)} \geq (1 - \varepsilon) r_c^{(k)}$, preventing total elimination of the category from future review.

The state of the loop at cycle k is the tuple $S^{(k)} := (\mathcal{I}^{(k)}, (r_c^{(k)})_{c \in \mathcal{C}}, (n_c^{(k)})_{c \in \mathcal{C}})$.

6.2 Conjecture C-T3 (Learning-Loop Convergence)

Conjecture 1 (Learning-Loop Convergence). *Assume:*

- (i) Stationary task distribution: *the bug-instance distribution per cycle is i.i.d. over a stationary law.*
- (ii) Conservative trigger: $\theta \geq 2$.
- (iii) Bounded injection: $M < \infty$.
- (iv) Weak suppression: $\varepsilon \in (0, 1)$.

Then the state process $(S^{(k)})_{k \in \mathbb{N}}$ converges almost surely as $k \rightarrow \infty$ to a finite stationary set $\mathcal{S}^$, in the sense that*

$$\exists k^*, \forall k \geq k^* : S^{(k)} \in \mathcal{S}^*.$$

Proof sketch. The conservative trigger ($\theta \geq 2$) ensures that each category accumulates at least θ observations before injection; under stationary law, $r_c^{(k)}$ converges to a population miss rate r_c^* as $k \rightarrow \infty$ by SLLN. Bounded injection ($M < \infty$) caps the size of $\mathcal{I}^{(k)}$. Weak suppression ($\varepsilon < 1$) prevents miss-rate collapse to 0, so categories remain comparable in subsequent cycles. The selection rule (top- M by r_c) stabilizes once $r_c^{(k)}$ are close enough to r_c^* , *provided* that the population miss rates of the categories at the top- M boundary are strictly separated (formal assumption: $\min_{c \in \text{boundary}(\mathcal{I}^*)} |r_c^* - r_{c'}^*| \geq \eta > 0$ for some fixed gap η , otherwise top- M ranking among tied categories oscillates indefinitely). Convergence to a finite $\mathcal{S}^*$ then follows from the finiteness of $\mathcal{C}$ and M . $\square$

Strict-separation assumption. The proof sketch assumes that no two categories at the top- M boundary have identical population miss rates. Without this gap, the loop would still converge in distribution but not point-wise; we adopt the stronger assumption because it matches the intended operational setting (categories with distinct semantic types).

Monte-Carlo simulation: on 30 Monte-Carlo runs with $|\mathcal{C}| = 7$, $M = 5$, $\theta = 2$, $\varepsilon = 0.3$, and i.i.d. stationary task distribution, the loop converges within $k^* \leq 8$ cycles in 30/30 runs (Phase 4 simulation).

6.3 LLM-grounded counterpart

The conjecture is about a simulation; the operationally interesting question is whether *LLM detection improves* when the loop’s output $\mathcal{I}^{(k)}$ is injected into the reviewer prompt. We formalize:

Let $\hat{r}_c^{\text{LLM}}(\mathcal{I})$ denote the empirical miss rate of an LLM warm reviewer on TEST-set bugs in category c , when the prompt is augmented with the injection set $\mathcal{I}$. The LLM-grounded version of C-T3 predicts

$$\hat{r}_c^{\text{LLM}}(\mathcal{I}^*) < \hat{r}_c^{\text{LLM}}(\emptyset) \quad \text{for } c \in \mathcal{I}^*, \quad (18)$$

i.e., injection of the converged set reduces miss rate on categories in the set.

6.4 Phase-5 falsification: abstract-name injection

We test Equation (18) with a TRAIN/TEST split ($n_{\text{TRAIN}} = 99$, $n_{\text{TEST}} = 101$ bugs). Top-5 categories by TRAIN miss rate are extracted: $\mathcal{I}_{\text{P5}}^* = \{\text{off-by-one, logic, currency, atomicity, exception}\}$. The warm-Codex prompt is augmented with the 5 category names plus a 1-line description each. Reviewer runs on the TEST set ($n = 92$ paired with baseline).

Result: baseline rate 75.0%, injected rate 68.5%, $\Delta = -6.5$ pp, paired McNemar one-sided $p = 0.95$ against H_1 : “injected > baseline”. **Falsifies** Equation (18).

6.5 Phase-5 E3v2: concrete-example injection

To rule out the format-quality alternative (“pattern names are too abstract”), we replace category names with *concrete missed-bug summaries* drawn from TRAIN. Per category, 2–3 concrete examples (e.g., for **logic**: “ $(1 + r/n)$ replaced with $(1 - r/n)$ in compound-interest formula”). 101 codex review calls on the same TEST set; $n = 92$ paired with baseline.

Result: baseline rate 75.0%, v2 injected rate 66.3%, $\Delta = -8.7$ pp, paired McNemar one-sided $p = 0.97$. Concrete examples *do not rescue* the injection effect.

Per-category descriptive deltas (exploratory, underpowered):

Category	n	baseline	v2	Δ
atomicity	4	75.00%	25.00%	−50 pp
currency	8	62.50%	100.00%	+37.5 pp
exception	9	88.89%	88.89%	0 pp
logic	23	73.91%	52.17%	−22 pp
off-by-one	12	83.33%	75.00%	−8 pp

We are explicit that atomicity ($n = 4$) is far too small for a reliable −50 pp interpretation; this is descriptive, not inferential. Per-category Wilson CIs are wide on all rows.

6.6 Simulation-to-LLM gap

The conjunction of:

- C-T3 converges almost surely in simulation (30/30 runs);
- Two pre-registered LLM-grounded injection formats both fail to improve detection at $n = 92$ paired, $p \in [0.95, 0.97]$;
- Concrete-example injection actively under-performs on 2/5 top categories (exploratory);

documents what we call the *simulation-to-LLM gap*: theoretical convergence properties of pattern-extraction loops do not transfer to measurable detection improvements in real LLM workflows at the tested n under either tested format. One concrete-example design does not rule out all format-quality alternatives (better example selection, quantity, placement, retrieval matching, abstraction level) and the negative finding may reflect prompt-budget dilution, attention competition, or anchoring on shown shapes.

C-T3 status: the simulation result stands (Conjecture 1 is unfalsified at the simulation level). The LLM-grounded version is robustly falsified under the two tested formats at the tested sample size; broader injection-format exploration is a Phase-12 desideratum.

7 Experiments

We report the eleven experimental phases in chronological order. Each phase has a pre-registered protocol (frozen before any reviewer call) and computational validation. Raw artefacts are committed at the paths indicated.

7.1 Phase 1–3: pilot ($n = 120$ trajectories, 4 frontier models)

Models: OpenAI Codex gpt-5.5, Claude Sonnet 4.6, Claude Opus 4.7, Gemini 2.5 Flash. **Tasks:** T1 (compound-interest with overflow handling) and T2 (200 single-line injected bugs across 7 categories in a finance domain). $n = 15$ trajectories per cell, 8 cells. Conditions: free ($\mathcal{P}_0$) vs constrained ($\mathcal{P}$, with spec-first + TDD + asymmetric reviewer).

Pilot finding (negative): the Shannon-entropy formulation of $\mathcal{V}$ used in v0.1 of the framework returns $\eta < 0$ under uniform-failure collapse, because $H(\mathcal{P}_0) < H(\mathcal{P})$ when both distributions concentrate on failure outcomes but $\mathcal{P}$ has more variability in compliance components. Switched to expected utility $\mathbb{E}[u]$ plus eFOSD, validated in Phase 4.

7.2 Phase 4 ($n = 120$): E1 validation, expected utility + eFOSD

Result: $\Delta\mathbb{E}[u] \in [+0.59, +0.78]$ across the 8 cells; eFOSD verified on 80/80 thresholds; $G \in [0.21, 1.11]$. Phase-4 also runs the 2×2 factorial (prompt-asymmetry $\times$ model-diversity) on $n = 200$ bugs $\times$ 7 reviewer roles = 1,400 LLM reviews:

	Single model (Codex)	4 models with cold-per-role
Symmetric prompts	symm-mono $q = 22.91\%$	symm-multi $q = 18.74\%$
Asymmetric prompts	asymm-codex $q = 16.55\%$	asymm-multi $q = 12.28\%$

Paired McNemar (symm-mono vs asymm-multi): $p = 0.0013$, Bonferroni-significant under all three parse-failure policies (strict, lenient, mid). Mechanism: $\bar{\rho}: 0.62 \rightarrow 0.29$ (Section 5).

7.3 Phase 5 ($n = 27 + n = 92$): L1 cross-domain falsification and C-T3 LLM-grounded falsification

L1 cross-domain falsification. The cold contract-first reviewer is re-deployed on `bankcheck.py` ($n = 27$ multi-line bugs across 10 categories) but with *finance contracts* (not domain-aligned). All 5 pre-registered contrasts return paired McNemar $p \geq 0.5$. Cold reviewer detection collapses to 11% on bankcheck. **Verdict:** the empirical mechanism of L1 is domain-bound; the mathematical bound continues to hold.

C-T3 LLM-grounded falsification. TRAIN/TEST split (99/101), top-5 categories injected via pattern names + 1-line descriptions, $n = 92$ paired. Result: $\Delta = -6.5$ pp, paired McNemar $p = 0.95$ against H_1 “injected $>$ baseline”. **Verdict:** simulation-derived injection strategy does not improve LLM detection at the tested sample size.

7.4 Phase 6: Cold reviewer recovery via auto-extracted contracts

To recover from the Phase-5 cross-domain collapse, contracts are *auto-extracted* from the bankcheck reference module via a single Codex call. Cold reviewer detection rises from 11% (misaligned) to 73% (auto-aligned) on the same bug suite. Component-level transfer recovered.

7.5 Phase 7: Second domain (JSON parser)

Cold reviewer with auto-extracted JSON-parser contracts vs cold reviewer with bankcheck (categorically mismatched) contracts: $n = 15$ bugs, paired McNemar $p = 0.0078$, $\Delta = +73$ pp. Bonferroni-significant. Recovery is robust to domain change.

7.6 Phase 8: Three hand-written domains (multi-domain pilot)

Expression evaluator, regex compiler, HTTP header parser. Bug-injection yield 44% (16/36 attempted via text-substitution), per-domain $n \in \{5, 6\}$, individually non-significant; Fisher’s

combined over P7 + P8 = 4 tests: $p = 0.021$ (below $\alpha = 0.05$, but *not* below pre-registered primary $p < 0.001$). Recorded as *partial multi-domain generalization, underpowered*.

7.7 Phase 9: Real Python stdlib codebases

Three stdlib codebases via AST mutator (Definition 1, yield 100%): `csv.py` (451 LOC), `urllib/parse.py` (1246 LOC), `json/decoder.py` (356 LOC). 30 AST mutations per domain, 7 reviewer roles, 630 Codex review calls.

Pre-registered metric results (Codex-only, strict unparsed-as-miss). *Primary test*: cluster-robust paired permutation on the within-domain bug list (treats bugs within a single module as a cluster, $n_{\text{perm}} = 20,000$); *secondary*: McNemar exact for comparability.

Domain	Cold	Cold-mm	Δ	Perm 95% CI on Δ	Perm p (1-sided)
csv_dom	43.3%	0.0%	+43.3 pp	[+26.7, +60.0] pp	$2 \cdot 10^{-4}$
urllib_dom	50.0%	0.0%	+50.0 pp	[+33.3, +66.7] pp	$< 5 \cdot 10^{-5}$
jsondec_dom	43.3%	0.0%	+43.3 pp	[+26.7, +60.0] pp	$1 \cdot 10^{-4}$

McNemar exact (secondary, $\alpha = 0.010$): $1.2 \cdot 10^{-4} / 3.1 \cdot 10^{-5} / 1.2 \cdot 10^{-4}$. The permutation p is more conservative by $\sim 2\text{--}3\times$, reflecting within-module clustering that McNemar ignores. The qualitative finding is robust: 3/3 domains pass at $\alpha = 0.010$ under both tests. The pre-registered absolute threshold “cold > 50%” fails on stdlib (0/3 strict, Wilson CIs all straddle 50%).

Cross-family on csv_dom (60 additional mismatched calls): all 3 families pass cluster-robust permutation at $\alpha = 0.010$: Codex $\Delta + 43.3$ pp (CI₉₅: [+26.7, +60.0], perm $p = 2 \cdot 10^{-4}$); Opus $\Delta + 26.7$ pp (CI₉₅: [+13.3, +43.3], perm $p = 4 \cdot 10^{-3}$); Gemini 3.1 Pro $\Delta + 60.0$ pp (CI₉₅: [+43.3, +76.7], perm $p < 5 \cdot 10^{-5}$). Even the smallest Δ (Opus) has a 95% CI strictly above zero.

Cross-family absolute detection on csv and jsondec (Codex / Opus / Gemini 3.1 Pro): csv 43% / 30% / 63%, jsondec 43% / 40% / 87%. Cross-family spread 33–47 pp.

7.8 Phase 10: Third-party Python libraries (3 domains $\times$ 3 families $\times$ 2 conditions $\times$ 30 bugs)

Three third-party Python modules selected for varying training-data presence: `dateutil/relativedelta.py` (599 LOC, high pop.), `parsy/__init__.py` (719 LOC, low pop.), `chardet/chardistribution.py` (233 LOC, medium pop.). AST mutator with compile-only validation (third-party modules require their own package context). 90 AST mutations, 540 reviewer calls.

Pre-registered cells ($9 = 3 \times 3$):

Domain	Family	Cold strict	Wilson 95%	Cold-mismatched	McNemar p
dateutil	Codex	93.33%	[78.7, 98.2]	0.00%	$< 10^{-6}$
	Opus 4.7	96.67%	[83.3, 99.4]	3.33%	$< 10^{-6}$
	Gemini 3.1 Pro	96.67%	[83.3, 99.4]	10.00%	$< 10^{-6}$
parsy	Codex	53.33%	[36.1, 69.8]	0.00%	$1.5 \cdot 10^{-5}$
	Opus 4.7	60.00%	[42.3, 75.4]	10.00%	$6.1 \cdot 10^{-5}$
	Gemini 3.1 Pro	66.67%	[48.8, 80.8]	6.67%	$2.4 \cdot 10^{-4}$
chardet	Codex	96.67%	[83.3, 99.4]	0.00%	$< 10^{-6}$
	Opus 4.7	96.67%	[83.3, 99.4]	3.33%	$< 10^{-6}$
	Gemini 3.1 Pro	93.33%	[78.7, 98.2]	0.00%	$< 10^{-6}$

Verdict: 9/9 cells pass cluster-robust permutation at $\alpha = 0.010$ (range $p \in [10^{-4}, < 5 \cdot 10^{-5}]$, all 95% CIs on Δ strictly above zero, smallest CI lower bound +13.3 pp for P9 csv Opus); 9/9 cells pass the point-estimate absolute threshold (range 53–97%); 6/9 cells pass the threshold under interval evidence (parsy 3 cells straddle 50%).

Table 1: C1 cross-phase pooled estimates. The mixed-effects $\hat{\beta}_{\text{cond}}$ is the primary v1.4 estimate; the block-bootstrap pooled Δ is the matched comparator; the legacy Fisher’s combined McNemar χ^2 is reported with the cross-phase model-version caveat of Section 8.4.

Estimator	Point estimate	95% interval / p
Mixed-effects $\hat{\beta}_{\text{cond}}$ (logit, primary)	+4.20	[+3.96, +4.45]; $z = +33.4$; $p_{\text{Wald},2} < 10^{-200}$
ME mean predicted $\hat{\Delta}$ across 14 cells	+68.7 pp	—
Approx. ELBO-LRT on joint cond block, $\text{df} = 4$	$D \approx 525$	$p \approx 3 \cdot 10^{-112}$
Block-bootstrap pooled Δ (comparator)	+67.4 pp	[+55.2, +79.5] pp
Fisher’s combined McNemar $\chi^2(14)$ (legacy)	167.72	$p \approx 0$ (caveat: Section 8.4)

Cross-phase pooled re-analysis (v1.4 primary). From v1.4 onwards the primary cross-phase inference is the bug-level mixed-effects logistic regression of Section A.5, fit on the 14-cell long-format export ($N = 840$ bug-level rows, 420 within-cell bug clusters, 6 module clusters). The condition main effect is $\hat{\beta}_{\text{cond}} = +4.20$ on the logit scale (95% Wald CI [+3.96, +4.45]; $z = +33.4$; two-sided Wald $p < 10^{-200}$); the approximate ELBO-based likelihood-ratio test on the joint {cond, cond $\times$ opus, cond $\times$ gemini, cond $\times$ libclass} block (4 df) yields $D \approx 525$ and $p \approx 3 \cdot 10^{-112}$. The cluster-robust block bootstrap over the 14 cells is retained as the matched comparator and gives observed pooled $\Delta = +67.4$ pp with 95% CI [+55.2, +79.5] pp; the mean predicted marginal $\hat{\Delta}$ across the 14 cells under the mixed-effects model is +68.7 pp (Table 1). Fisher’s combined Codex-only over P7 + 3 P9 + 3 P10 = 7 McNemar tests ($\chi^2(14) = 167.72$, $p \approx 0$) is reported as a legacy comparator only: it both treats within-cell observations as independent (inflating significance) and assumes between-test independence that is partially compromised by cross-phase model-version changes (see Section 8.4).

Per-cell three-way comparison. All 14/14 cells qualitatively agree at $\alpha = 0.025$ (one-sided): McNemar exact, cluster-robust paired permutation, and the mixed-effects predicted marginal contrast all flag cold > mismatched with intervals excluding 0. The full per-cell table is reported in the appendix (Table 3).

Cross-family spread on third-party: 3.3–13.3 pp (vs 33–47 pp in P9 stdlib sample).

7.9 Phase 11: Provenance-label experiment

Phase 11 ran in two passes. The v1.3 pilot used 30 paired Codex calls per side (60 total); the v1.4 saturation replication extended this to all available AST mutations on the same two frozen reference modules: 53 paired pairs for csv (csv_module.py) and 60 for chardet (chardistribution_module.py). The v1.4 replication is pre-registered in docs/paper/v1.4/preregistration (frozen 2026-05-11 17:00) with Amendment A1 (preregistration-p11-replication-amendment-A1.md, frozen 2026-05-11 21:00) authorising a saturation variant after pre-flight enumeration established that $n = 100$ per cell was infeasible on the frozen modules (csv: 53 valid PRIORITY-1 mutations; chardet: 60). Amendment A1 also relaxes the operator-mix-proportion match (BOR is exhausted in v1.3 on both modules); per-operator counts are reported in the manifest. The 1-line cover stories and the contract+prompt splices are byte-identical to v1.3 across all conditions; only the leading provenance statement changes between truthful and relabeled. Power for $\Delta = 10$ pp at $\alpha = 0.025$ (one-sided Bonferroni-per-cell) under the saturation n is 0.55–0.65, an improvement over v1.3’s ~ 0.30 –0.35 but below the parent pre-reg’s planned 0.80.

Result (v1.3 pilot vs v1.4 saturation replication):

Cell	v1.3 $n = 30$, Δ & McNemar p	v1.4 $n = 53/60$, Δ & McNemar p	v1.4
csv (stdlib $\rightarrow$ 3p label) H1a: rel > truth	+6.7 pp, $p = 0.3438$	+0.0 pp, $p = 0.6128$	0
chardet (3p $\rightarrow$ stdlib label) H1b: rel < truth	−10.0 pp, $p = 0.1250$	−3.3 pp, $p = 0.3125$	0

At the pre-registered Bonferroni-per-cell $\alpha = 0.025$, neither v1.4 cell rejects the null (csv McNemar $p = 0.6128$; chardet $p = 0.3125$). Bootstrap 95% CIs on Δ are $[-13.2, +13.2]$ pp for csv and $[-10.0, +3.3]$ pp for chardet. **Decision rule: v1.3 conclusion stands at higher power.** The residual label-only effect is bounded by the v1.4 CI upper edges, i.e. at most ~ 13 pp on csv (relabel-as-third-party direction) and ~ 3 pp on chardet (relabel-as-stdlib direction), tighter than v1.3’s ± 18 pp. The same data broken into the 30 v1.3-reused bugs and the new bugs gives consistent point estimates ($\Delta = 0.0$ pp on csv subgroups; chardet v1.3-reused $\Delta = -6.7$ pp, new $\Delta = 0.0$ pp); see `experiments/p11_replication/results.json`.

Halt #6 (pre-registered): a v1.4 result that significantly reverses the v1.3 directional trend would force a §7.9 reframing. Neither cell is significant in v1.4 and the point estimates do not flip past zero on csv ($\Delta = 0.0$) or on chardet (Δ remains negative); halt #6 is *not* triggered.

Drift diagnostic: the 30 v1.3-reused bugs were re-run truthful concurrently with v1.4 (separate output subdirectory). csv truthful detection rate moved from 43.3% (v1.3, on disk) to 53.3% (v1.4 re-run; McNemar $p = 0.19$, NS) — a +10 pp mid-strength Codex drift on csv. chardet rates were stable (96.7% both runs). The contemporaneous v1.4 main run on the same 30 reused csv bugs gave 60.0% truthful = relabeled — within the drift envelope. A KS two-sample test on per-bug truthful detection (v1.3-reused vs v1.4-new) reports $D = 0.165$ and $p = 0.835$ on csv (same-distribution sanity passes; the v1.4-new bugs are not measurably harder than the reused ones) and $D = 0$ on chardet (identical means; the chardet baseline is saturated).

Anti-stdlib carelessness as the dominant mechanism: not supported (at $n = 53/60$ per side). Label-only effects within roughly ± 13 pp on csv and ± 10 pp on chardet are below the saturation- n power threshold and cannot be ruled out by this experiment; only the much larger “label as dominant cause of the ~ 53 pp sample gap” interpretation is rejected. Operator-mix deviation under Amendment A1 (BOR exhausted) is documented; the same-distribution KS sanity is consistent with the deviation not introducing a measurable difficulty shift on csv. Methods, raw outputs, and the analyzer that produced these numbers are in `experiments/p11_replication/` on branch `v1.4/exp-p11-rep`.

7.10 Phase 12: Naturalistic bug benchmark on csv.py ($n = 8$)

Motivation. Phases 5–11 establish the C1 specificity finding on AST single-mutation bugs (AOR/ROR/BOR). Peer reviewers in successive rounds asked whether the contract-aligned advantage transfers to *naturalistic* bugs harvested from real issue history. Phase 12 tests this on `Lib/csv.py`, the stdlib module with the lowest cold-detection rate in Phase 9 and therefore the most informative comparator. The pre-registration is at `docs/paper/v1.4/preregistration-extend.md` (frozen 2026-05-11).

Harvest. We enumerated all 80 commits ever touching `Lib/csv.py`, applied the inclusion criteria (single-function-scoped, ≤ 30 LOC net change, no API change, behavioural fix, reversal patch applies cleanly to the v3.12 reference and produces an importable Python 3 source), and excluded documentation-only, refactor-only, perf-only, and Python-2-syntax-only fixes. Eight bugs survived (seven via direct `patch -R` on the merge commit, one via line-level manual port). This is below the pre-registered target of 25–30; pre-registration risk #1 anticipates this case and mandates “proceed at n harvested with reduced power”. Power at $n = 8$ for $\Delta = 30$ pp, $\alpha = 0.025$ paired one-sided is approximately 0.30; the test is severely under-powered and the per-family significance read is qualitative.

Method. Three reviewer families (Codex gpt-5.5, Claude Opus 4.7, Gemini 3.1 Pro Preview) review each bugged source under *cold* (auto-extracted `csv.py` contracts, byte-identical to the Phase 9 artefact) and *cold_mismatched* (the bankcheck contracts from Phase 6, byte-identical, deliberately wrong). Detection criterion is frozen at harvest: per bug, 5–9 `expected_detection_keywords` (function name + bug summary terms + cited error type); a reviewer detects iff any keyword (case-insensitive) appears in `bugs_found`. An out-of-band probe asks each reviewer whether they

have seen this bug in the CPython issue tracker; cite-rate is reported as a leakage diagnostic and does *not* enter the detection metric.

Per-family result.

Family	cold	mm	Δ	cluster-robust perm p	McNemar p
Codex gpt-5.5	75.0%	12.5%	+62.5 pp	0.0286	0.0312
Claude Opus 4.7	12.5%	0.0%	+12.5 pp	0.4974	0.5000
Gemini 3.1 Pro Preview	62.5%	0.0%	+62.5 pp	0.0308	0.0312

Per-family Bonferroni-corrected $\alpha = 0.025$. None of the three families crosses the threshold; Codex and Gemini sit just above it ($p \approx 0.03$). Under the looser family-wise envelope $\alpha = 0.075$, no family is significant either. **0/3 H1 replicates at the pre-registered threshold**, with the strong caveat that $n = 8$ paired permutations cannot reach $p < 0.025$ unless the entire effect is concentrated in one direction (here 5/8 discordants for Codex and Gemini in the cold-wins direction, 0/8 in the mm-wins direction).

Leakage diagnostic. CPython-issue-number cite-rate from the out-of-band probe: Codex 100.0% (**above pre-registered 30% caveat threshold**); Opus 0.0%; Gemini 3.1 Pro 12.5%. **Codex cites a CPython issue number on every bug:** its 75% cold-detection rate cannot be cleanly attributed to contract reasoning—training-set memorization fully explains it. Per pre-reg decision rule, the Codex result is reported with strong caveat. Opus probes always answer “NO”. Gemini cites 1/8 = 12.5% (below threshold but non-zero).

Naturalistic vs AST cold-rate on the same module (Mann-Whitney U, unpaired because bug sets differ; secondary H1 predicts naturalistic $\leq$ AST). Codex: 43.3% AST vs 75.0% naturalistic, $\Delta = -31.7$ pp, $p = 0.942$ (inversion aligns with the 100% leakage signal). Opus: 30.0% vs 12.5%, $\Delta = +17.5$ pp, $p = 0.162$ (direction as predicted, not significant). Gemini 3.1 Pro: 63.3% vs 62.5%, $\Delta = +0.8$ pp, $p = 0.483$ (indistinguishable).

Conclusion and interpretation. At $\alpha = 0.025$, C1 specificity does *not* replicate on naturalistic `csv.py` bugs in any of the three families (0/3). The Codex and Gemini point-estimates ($\Delta = +62.5$ pp) are large and in the predicted direction but cannot clear the Bonferroni threshold at $n = 8$ paired even when the effect is concentrated; the Codex signal is further confounded by 100% training-set leakage of CPython issue numbers, and the Opus signal is consistent with no specificity transfer at this n . The secondary H1 (naturalistic $\leq$ AST) is directionally supported only for Opus. The benchmark therefore reveals two residual concerns about the broader C1 result—small- n underpower and leakage on widely-discussed `stdlib` code—and motivates a Phase-13 desideratum: a leakage-controlled naturalistic benchmark on post-cutoff or non-CPython libraries.

7.11 Empirical summary

Eighteen-plus cells across four phases (P7 + 3 P9 + 9 P10 + provenance) confirm the C1 specificity component. The absolute-detection moderator(s) are code-intrinsic and unidentified (Phase-12 desiderata: contract informativeness audit, matched-LOC `stdlib`/third-party experiments, operator-mix balancing, no-contract baseline).

8 Threats to Validity and Discussion

We organize the discussion of validity threats following the Wohlin et al. template [25] (construct, internal, external, conclusion), and close with an explicit accounting of what this paper has and has not established. The ten failure modes named throughout the paper are listed under internal validity; the cross-phase model-version confounder is listed under conclusion validity.

8.1 Construct validity

The constructs measured in this paper are: (i) target utility $u(B) = \alpha c_{\text{corr}} + \beta c_{\text{rob}} + \gamma c_{\text{form}}$ with $\alpha + \beta + \gamma = 1$ and $\alpha + \beta > \gamma$ (the latter constraint ensures held-out components dominate over conformance, avoiding the conformance-without-semantics trap); (ii) the conformance/semantics split ($\text{Pred}_C, \text{Pred}_S$) operationalized by indicator predicates on V_C and V_S verdicts respectively (Equations (6) and (7)); (iii) the bug-detection criterion (line ± 2 or enclosing function plus operator keyword) used uniformly across Phases 4–11. Each of these is a measurement choice: a different utility weight scheme, a different held-out split, or a different detection-localization tolerance would produce different numerical values for the same trajectory data. The qualitative findings (eFOSD on 80/80 thresholds, $\bar{\rho}$ collapse from 0.62 to 0.29, Phase-5 cross-domain falsification, Phase-11 provenance falsification) are robust to the specific weight choices reported here; we have not, however, systematically explored the sensitivity to alternative construct operationalizations.

8.2 Internal validity: ten failure modes (operationally defined)

1. **Uniform-failure collapse.** $\mathbb{E}[u(B)] \approx 0$ under both $\mathcal{P}_0$ and $\mathcal{P}$ on a task neither can solve. Shannon-entropy formulation of $\mathcal{V}$ returns $\eta < 0$. *First observed in pilot Phase 1–3*; resolved by switching to $\mathbb{E}[u] + \text{eFOSD}$.
2. **Oracle leakage.** Failure of Equation (3): $Y_S(\tau) \not\leq h_t$. Operationally: Pred_S verdicts (test outputs, traces, hints) appear in observations o_t . *No explicit measurement* (would require auditing each o_t); flagged as the most dangerous failure mode and prevented by held-out splits.
3. **Symmetric-reviewer anchoring.** Three reviewers with identical prompts have high $\bar{\rho}_{ij}$. Operationally: $\bar{\rho}_{ij} > 0.5$ in the symm-mono condition. *Quantified Phase 4*: $\bar{\rho}_{\text{symm-mono}} = 0.62$.
4. **Model-strength ceiling.** Claude or Gemini may dominate Codex on a fixed task; a multi-model ensemble’s joint-miss reduction may be confounded with model-strength differences rather than error decorrelation. *Phase 4 controls this via the asymm-codex cell (single-model asymmetric prompts)*: $q_{\text{asymm-codex}} = 16.55\%$ already substantially below symm-mono 22.91%, confirming prompt asymmetry contributes independently of model diversity.
5. **Conformance without semantics.** $\mathbb{E}_{\mathcal{P}}[\text{Pred}_C] \gg \mathbb{E}_{\mathcal{P}}[\text{Pred}_S]$. The protocol is followed but artefacts fail held-out tests. *Quantified Phase 4*: 33–68% of $\mathcal{P}$ -condition runs depending on the chosen Pred_S robustness threshold.
6. **Off-by-one boundary blind spots.** A category-specific blind spot: reviewers preferentially miss boundary errors ($>$ vs $\geq$, $<$ vs $\leq$, index ± 1). Operationally identified by per-category detection rates. *Quantified Phase 4–5*: off-by-one bugs have the highest miss rate (25%) among the seven tested categories.
7. **Out-of-domain prompt collapse.** The cold contract-first reviewer drops from $\sim 90\%$ to 11% when given misaligned contracts. *Quantified Phase 5*; *recovered Phase 6+* via auto-extracted aligned contracts.
8. **Simulation-to-LLM gap.** Theoretical convergence in simulation does not transfer to LLM detection improvement. *Quantified Phase 5 (abstract-name injection, $p = 0.95$) and E3v2 (concrete-example injection, $p = 0.97$)*. Both injection formats fail at $n = 92$ paired.
9. **Sample-dependent reviewer-choice sensitivity.** Cross-family absolute-detection spread varies by sample: 33–47 pp on the P9 stdlib sample vs 3.3–13.3 pp on the P10 third-party

sample. *Mechanism unknown*: Phase 11 rules out the label/familiarity-prior candidate; code-intrinsic factors remain.

10. **Descriptive injection-anchoring effects (exploratory)**. Concrete-example injection underperforms baseline on 2/5 top categories (atomicity -50 pp, $n = 4$; logic -22 pp, $n = 23$) in E3v2. *Underpowered for inferential claims*; consistent with reviewer-anchoring on shown shapes.

8.3 External validity

We deliberately do not claim general applicability:

- **Tasks**: E1 validated on 2 tasks; L1 on 1 finance domain; C1 on 8 domains (2 hand-written + 3 stdlib + 3 third-party). **All Python**. No claim is made about transfer to other languages.
- **Models**: E1 on 4 models (Codex `gpt-5.5`, Claude Sonnet 4.6, Claude Opus 4.7, Gemini 2.5 Flash); L1 within the same set; C1 with 3 frontier families on Phase-10 (Codex `gpt-5.5`, Claude Opus 4.7, Gemini 3.1 Pro Preview). See Section 8.4 for the model-version confounder.
- **Sample sizes**: $n \in \{27, 30, 60, 92, 200\}$ bugs across phases. Per-domain power analyses are reported in the respective phase reports.
- **Bug-generation procedure**: Phases 4–11 use **AST-mutator-only** single-mutation edits produced by a deterministic mutator. Section 7.10 reports a naturalistic replication on $n = 8$ harvested CPython `csv.py` bugs (multi-line, inter-procedural): 0/3 families reach the Bonferroni-corrected threshold (Codex $p = 0.029$, Gemini $p = 0.031$, Opus $p = 0.50$); the Codex result is contaminated by 100% training-set leakage of CPython issue numbers in the out-of-band probe. Point-estimate Δ s (Codex $+62.5$ pp, Opus $+12.5$ pp, Gemini $+62.5$ pp) are large and in the predicted direction, but the harvest yield ($n = 8$, below the pre-registered target of 25–30) gives each per-family test very low power. Naturalistic C1 transfer at low n with the leakage caveat is *reported but not claimed*; it remains a Phase-13 desideratum (leakage-controlled, post-training-cutoff bugs, or non-CPython libraries).
- $\mathcal{V}$ is a **Lagrangian one-shot summary, not a full optimal-control problem**. Real cost curves have cliff effects (context window limits, batching effects, API pricing changes) that the one-dimensional λ does not capture; the break-even λ is computed in expected log-token units, not in real-dollar latency-weighted budgets.

8.4 Conclusion validity

Statistical methodology. In v1.4 we promote the mixed-effects logistic model (Section A.5, BinomialBayesMixedGLM with per-bug and per-module random effects and a library-class moderator on the `cond` $\times$ `libclass` interaction) to the primary cross-phase estimator over the 14 cells (P9 + P9 cross-family + P10), reporting $\beta_{\text{cond}} = +4.20$ ($z = +33.4$), $\sigma_{\text{bug}} = 0.72$, $\sigma_{\text{module}} = 0.77$, and a mean predicted $\hat{\Delta} = +68.7$ pp. The cluster-robust paired permutation ($n_{\text{perm}} = 20,000$) with percentile-bootstrap CIs is retained as the matched per-cell comparator and the block-bootstrap over cells (cross-phase pooled $\hat{\Delta} = +67.4$ pp, 95% CI $[+55.2, +79.5]$) as the matched aggregate comparator; McNemar exact is kept as a legacy comparator with the prior literature. All 14 cells agree qualitatively across the three procedures at $\alpha = 0.025$ (Table 3, Table 1). Full implementation and per-cell numbers are in Section A (`experiments/cluster_robust.py`, `cluster_robust_c1.py`, and `experiments/mixed_effects/fit.py`).

Cross-phase model-version confounder. The model lineup is not constant across phases. Phases 1–5 use Gemini 2.5 Flash; Phase 9 cross-family extension and Phase 10 introduce Gemini 3.1 Pro Preview. Codex gpt-5.5 and Claude Opus 4.7 are stable across all phases that use them. This is a real confounder for any cross-phase aggregation:

- **Fisher’s combined statistic** over P7 + 3 P9 + 3 P10 (Section 7.8) combines per-test p -values that are not strictly comparable: Phase 7–9 use older model snapshots in the supporting ensemble; Phase 10 uses Gemini 3.1 Pro. The headline $\chi^2(14) = 167.72$, $p \approx 0$ is therefore an aggregate of tests with slightly different underlying populations.
- **Within-Codex tests** are not affected (the Codex gpt-5.5 family is stable across phases).
- **Cross-family comparisons within a single phase** (P9, P10) are not affected.

We do not rerun Phases 1–5 with Gemini 3.1 Pro because the central claims of those phases (E1 expected utility on $\mathcal{P}_0$ vs $\mathcal{P}$, L1 within-finance reduction in joint-miss) are within-condition contrasts that do not require model parity across phases. The confounder is real but bounded; the central claims survive. We flag this explicitly and request that re-analyses by future authors recompute the cross-phase Fisher statistic using only Codex-stable data or only single-phase data.

Phase-12 power and leakage caveats. The naturalistic Phase-12 replication (Section 7.10) is conclusion-validity-limited on two fronts: (i) the strict-inclusion harvest yielded $n = 8$ bugs against a pre-registered target of 25–30, so the per-family paired permutation test is severely underpowered—even the largest observed point estimates ($\Delta = +62.5$ pp for Codex and Gemini) do not clear the Bonferroni-corrected $\alpha = 0.025$ threshold ($p = 0.029$ and $p = 0.031$ respectively); (ii) the Codex cold reviewer’s out-of-band probe shows 100% leakage of CPython issue numbers, indicating that its naturalistic detection is contaminated by training-set memorization and the corresponding Δ cannot be attributed to contract reasoning. Opus shows 0% leakage and Gemini 12.5%; both nonetheless fail the primary threshold for the underpowered- n reason. We treat Phase 12 as a strong invitation to replicate on leakage-controlled, post-training-cutoff naturalistic bugs and do not draw inferential conclusions from its point estimates.

8.5 What this paper has and has not established

Proven (mathematically): the joint-miss bound Equation (17) (routine; inclusion–exclusion), the eFOSD certification Lemma 1 (DKW), the simulation convergence of Conjecture 1.

Measured empirically within explicit scope: E1 ($n = 120$, 2 tasks, 4 models, eFOSD on 80/80 thresholds); L1 mechanism within the finance domain ($n = 200$ bugs, $\bar{\rho}: 0.62 \rightarrow 0.29$, paired McNemar $p = 0.0013$); C1 contract-aligned specificity in 18^+ cells across 3 reviewer families.

Empirically falsified or non-corroborated:

- L1 mechanism transfer to a non-finance domain *without re-tooled contracts* (**falsified**), Phase 5.
- C-T3 LLM-grounded improvement at $n = 92$ paired (**no improvement detected; we report absence of detected improvement, not evidence of absence of any effect smaller than our power threshold**) under abstract-name injection (Phase 5) and concrete-example injection (E3v2).
- “Anti-stdlib carelessness” as the dominant mechanism behind the P9–P10 sample gap (**rejected**), Phase 11.

Note on C-T3. Two pre-registered injection formats failed to detect improvement at $n = 92$. This does *not* establish that no injection format works; the design space (example selection, retrieval matching, abstraction level, placement, prompt-budget rebalancing) is large and a single format cannot rule out all alternatives. The defensible statement is: *at this sample size and under the two formats tested, the simulation-derived injection strategy did not improve LLM detection*. Phase 12 desideratum: power analysis to bound the residual undetectable effect size.

Open:

- The simulation-to-LLM gap mechanism.
- The dominant code-intrinsic moderator(s) of absolute cold-reviewer detection.
- Generalization to non-Python languages, naturalistic bugs, and lower-popularity libraries.

8.6 Internal-review trail and frozen-pre-registration discipline

We adopted a methodological scaffold of *frozen pre-registrations followed by adversarial internal review at each major version*, with all critique documents and applied reframings committed in the companion repository under `docs/paper/v0.1/` and `docs/paper/v1.4/`. The discipline has three components: (i) every experimental phase has a pre-registration committed before any reviewer call (verifiable via git-log timestamp); (ii) every major manuscript version was passed through an adversarial-review round whose verbatim critique was committed before the applied reframing; (iii) when a reviewing round produced a REJECT or REQUEST_CHANGES, the claim language was softened rather than the data being re-examined. The most consequential reframings produced by this trail are summarized in Table 2; we record them not as anecdote but as evidence that the framework’s claims have already survived an honest first-pass adversarial round.

File-path and reproducibility details are consolidated in the conclusion’s reproducibility section (Section 9) rather than duplicated here.

9 Conclusion

We presented a measurement framework—not a general theory—for LLM coding agents under procedural constraints. Three formal statements anchor the framework: E1 (Expected-Utility Dominance + eFOSD), L1 (Asymmetric-Quorum Joint-Miss Bound), and C-T3 (Learning-Loop Convergence). A fourth empirical claim, C1 (Cold reviewer transfer), replicates contract-aligned specificity across 4 phases, 18^+ cells, 3 reviewer families, and 8 domains (2 hand-written, 3 stdlib, 3 third-party).

The honest empirical record is:

- **E1** validated on $n = 120$: $\Delta\mathbb{E}[u] \in [+0.59, +0.78]$, eFOSD on 80/80 thresholds, $G \in [0.21, 1.11]$.
- **L1** validated within finance ($n = 200$, paired McNemar $p = 0.0013$); cross-domain transfer *falsified* (Phase 5, `bankcheck.py`, $p \geq 0.5$ in all 5 contrasts); *recoverable* via auto-extracted aligned contracts (Phase 6+).
- **C-T3** simulation converges 30/30; LLM-grounded *falsified* under two injection formats ($p = 0.95$ and 0.97).
- **C1** specificity replicates in all 18^+ tested cells. Absolute-detection moderator(s) are code-intrinsic and unidentified; *anti-stdlib carelessness not supported as the dominant mechanism* by Phase-11 provenance-label experiment (effects within ± 10 pp residual undetectable at the tested n).

Ten failure modes are operationally defined and (where possible) quantified.

Table 2: Most consequential reframings produced by the iterative adversarial-review trail.

Round	Claim modified	Before → After
v0.7 → v0.8	Phase-9 cold threshold	“cold > 50% threshold PASSES” → “absolute threshold falsified on real stdlib” (lead with falsification).
v1.0 → v1.0.5	Stdlib-bias direction	“stdlib-bias hypothesis REVERSED, anti-stdlib carelessness” → “sample-level difference; library class is a candidate moderator”.
v1.0.5 → v1.1	Anti-stdlib carelessness mechanism	“REJECTED” → “not supported as dominant mechanism (post-P11 provenance falsification)”.
v1.1 → v1.2	P1/P2 status, title, abstract, §7, C-T3 framing	P1 Proposition → Empirical Claim E1; P2 Proposition → Lemma L1 (now Eq. 17); title reordered to lead with “Measurement Framework”; abstract trimmed $\sim 50\%$; §7 status chapter deleted; C-T3 reframed from “robust to two tested formats” to absence-of-detected-improvement at $n = 92$.
v1.2 → v1.3	CMDP scope, primary statistical test	§3 CMDP reduced to load-bearing primitives (full construction moved to Appendix A.2; <i>math washing</i> explicitly acknowledged); primary inference changed from per-cell McNemar exact to cluster-robust paired permutation ($n_{\text{perm}} = 20,000$) with percentile-bootstrap CI for Δ ; cross-phase pooled $\Delta = +67.4$ pp, 95% CI [+55.2, +79.5] pp.

9.1 Phase-12 desiderata

We name what we cannot answer with the present data:

1. **Contract informativeness audit.** A blinded human-rater audit of auto-extracted contracts across the 8 domains, scored on completeness, specificity, and operator-level invariant clarity. Tests whether contract quality varies systematically across library classes.
2. **Matched-LOC stdlib/third-party experiment.** Pair stdlib and third-party modules at fixed LOC (e.g., 300 LOC from each), with operator-mix balancing of AST mutations. Tests whether the P9-P10 sample gap survives strict matching.
3. **No-contract baseline.** Cold reviewer prompt without any contract document, only the cover-story preamble. Tests whether contracts add information or only re-format the bug-detection task. Currently we have warm/skeptic/generic comparators but no pure no-contract control.
4. **Shuffled-within-domain contracts.** Cold reviewer with contracts shuffled at function level (function A ’s contract attached to function B in the same module). Tests whether contracts are useful as *structure* or only as *aligned content*.
5. **Naturalistic bug benchmark.** Replace AST single-mutation bugs with naturalistic bugs

harvested from real GitHub commit histories. Most likely outcome: substantial detection drop across all conditions; relative effects may be preserved.

6. **Cross-language generalization.** Replicate AST mutator on TypeScript, Rust, and Go reference modules. Tests whether C1 specificity is Python-bound.
7. **Multi-call self-consistency.** $5\times$ sampling per reviewer per bug, majority vote. Tests stability of the 93–97% Gemini 3.1 Pro detection rates and whether quorum-style aggregation within a single family approaches the asymm-multi performance.
8. **Reviewer-family ensemble on third-party (L1 cross-family).** Asymm-multi {Codex + Opus + Gemini 3.1 Pro} on the P10 dataset. Tests whether L1’s joint-miss reduction holds across families on real third-party code. Data is already collected; the analysis is an exercise.
9. **Provenance-label power analysis.** Phase 11 was underpowered ($n = 30$ per cell; power to detect ≥ 20 pp label effect was high but ≤ 10 pp effects would not be detected). Re-run at $n = 100$ per cell to bound the residual label effect.
10. **Mixed-effects modeling.** Replace cell-wise marginal McNemar with a hierarchical logistic model on the bug-level binary outcomes: $\text{logit } P(\text{detect}) = \beta_0 + \beta_{\text{cond}} + \beta_{\text{family}} + \beta_{\text{domain}} + \text{bug}_i$. Tests cross-condition / cross-family / cross-domain interactions with correct uncertainty quantification.

9.2 Final stance

We provide a measurement methodology with explicit boundaries of validity, explicit recovery mechanisms, and a ten-mode failure taxonomy. We deliberately distinguish what is proved, what is measured, what is conjectured, what is empirically falsified, and what is partially recovered under explicit conditions. Where claims overreached in earlier drafts, we report the verbatim adversarial-review critique and the applied reframing, so the trail of revisions is itself reproducible.

The contribution is not a new prompting technique or a benchmark; it is a discipline of separation between proved, measured, conjectured, and falsified.

Reproducibility statement. All numerical claims are recomputed from raw artefacts by validation scripts in the companion repository (`experiments/p{N}_*/analyze_*.py`). Pre-registration documents are committed before any reviewer call. Reviewer raw outputs, AST mutator code, auto-extracted contracts, and reference modules are committed at indicated paths.

A Statistical Methods Appendix

A.1 Paired McNemar exact one-sided test [17]

Given a paired contingency table on n matched bug instances:

	B catches	B misses
A catches	both	A-only = c
A misses	B-only = b	both

We test $H_1 : P_A > P_B$ against $H_0 : P_A = P_B$ using the exact binomial:

$$p_{\text{one-sided}} = \frac{1}{2^{b+c}} \sum_{k=0}^b \binom{b+c}{k}, \quad (19)$$

which is the probability under H_0 of observing b or fewer “A-only” outcomes out of $b + c$ discordant pairs. Discordant pairs with $b = c = 0$ are treated as $p = 1$ (no evidence).

For multiple comparisons we use Bonferroni: family-wise $\alpha = 0.05$ across the 5 original P4 contrasts gives per-test $\alpha_{\text{Bonf}} = 0.010$. We carry this stricter threshold forward to Phases 9–11 as the per-domain significance criterion.

A.2 Wilson score 95% confidence interval [24]

For an empirical proportion $p = k/n$ we report the Wilson score CI:

$$p_{\pm} = \frac{1}{1 + z^2/n} \left[p + \frac{z^2}{2n} \pm z \sqrt{\frac{p(1-p)}{n} + \frac{z^2}{4n^2}} \right], \quad (20)$$

with $z = 1.96$ for the 95% level. Wilson is preferred over Wald-normal at small n and at boundary proportions because it does not collapse at $p \in \{0, 1\}$.

A.3 Fisher’s combined p -value [8]

For K independent p -values $p_1, \dots, p_K$ from independent tests of the same null,

$$\chi_{\text{Fisher}}^2 = -2 \sum_{k=1}^K \log p_k \sim \chi_{2K}^2 \text{ under } H_0. \quad (21)$$

We use this to combine the per-domain McNemar p -values from P7 + 3 P9 + 3 P10 = 7 tests (Codex-only). At-cell independence holds because each test is on a disjoint bug set with a disjoint reviewer ensemble.

Caveat (cross-phase model-version): the cross-phase combination is *not* strictly between independent identically-conditioned tests, because the supporting model lineup changes between Phase 5 / 7 (Gemini 2.5 Flash) and Phase 9 cross-family / Phase 10 (Gemini 3.1 Pro Preview). See Section 8.4 for the discussion: within-Codex tests are unaffected (Codex `gpt-5.5` is stable), but the Fisher’s combined headline reported in Section 7.8 is not strictly homogeneous across phases. We retain it as a comparator with prior reporting practice, not as the primary inference; the primary cross-phase summary in v1.3 is the block-bootstrap over cells (Section 7.8).

A.4 DKW concentration for eFOSD

The Dvoretzky–Kiefer–Wolfowitz inequality [6] gives a finite-sample bound:

$$\mathbb{P} \left[\sup_t |\hat{F}_n(t) - F(t)| > \epsilon \right] \leq 2e^{-2n\epsilon^2}. \quad (22)$$

By a union bound over a finite grid of K thresholds, eFOSD certified at slack ϵ on the grid certifies population FOSD on those thresholds with confidence $1 - 4Ke^{-2n\epsilon^2}$. We use this as a sanity check only; eFOSD is reported on the empirical grid without claiming the population extension.

A.5 Mixed-effects logistic model (primary) and comparators

Within a single bug-injection campaign, mutations on the same reference module are not independent: they share the module’s structural features (common idioms, operator distributions, function boundaries). The same bug instance is also re-presented to multiple reviewer families in the cross-family cells. McNemar exact treats every paired observation as independent and a cluster-robust paired permutation accounts for within-cell dependence but not for the cross-cell bug-level / module-level dependence structure. From v1.4 onwards we therefore promote a

mixed-effects logistic regression to primary inference on C1 cells, retain the cluster-robust paired permutation as the matched comparator, and keep McNemar exact only as a legacy comparator with prior reporting practice.

Model specification (frozen, pre-analysis). Let $y_i \in \{0, 1\}$ denote the strict-policy detection outcome of the i -th bug-level row in the long-format export of $N = 840$ rows (14 cells $\times$ 30 bugs $\times$ 2 conditions). With $\text{cond}_i \in \{0, 1\}$ for mismatched/cold, $\text{fam}_i \in \{\text{codex}, \text{opus}, \text{gemini31}\}$, $\text{libclass}_i \in \{\text{stdlib}, \text{third_party}\}$,

$$\begin{aligned} \text{logit } \mathbb{P}(y_i = 1) = & \beta_0 + \beta_{\text{cond}} \text{cond}_i + \beta_{\text{opus}} \mathbf{1}[\text{fam}_i = \text{opus}] + \beta_{\text{gem}} \mathbf{1}[\text{fam}_i = \text{gemini}] \\ & + \beta_{\text{lib}} \mathbf{1}[\text{libclass}_i = \text{tp}] + \beta_{\text{c} \times \text{opus}} (\text{cond}_i \cdot \mathbf{1}[\text{opus}]) + \beta_{\text{c} \times \text{gem}} (\text{cond}_i \cdot \mathbf{1}[\text{gem}]) \\ & + \beta_{\text{c} \times \text{lib}} (\text{cond}_i \cdot \mathbf{1}[\text{tp}]) + u_{\text{bug}[i]} + u_{\text{module}[i]}, \end{aligned} \quad (23)$$

with $u_{\text{bug}} \sim \mathcal{N}(0, \sigma_{\text{bug}}^2)$ over the 420 per-cell bug clusters, $u_{\text{module}} \sim \mathcal{N}(0, \sigma_{\text{module}}^2)$ over the 6 modules, and reference levels $\text{cond} = \text{mismatched}$, $\text{fam} = \text{codex}$, $\text{libclass} = \text{stdlib}$. The DKW / strict-separation assumption invoked elsewhere in the paper for the C-T3 simulation is *not* required for this model.

Estimation. The model is fit by mean-field variational Bayes via `statsmodels.genmod.bayes_mixed_glm.BinomialBayesMixedGLM.fit_vb` [21] on the long-format CSV `experiments/mixed_effect`. fixed-effect posterior summaries are reported as $\beta \pm 1.96 \cdot \text{SD}$ (approximate Wald-style 95% interval) with two-sided $p = 2(1 - \Phi(|\beta/\text{SD}|))$. Per-cell predicted marginal contrasts $\hat{\Delta} = \hat{p}_{\text{cold}} - \hat{p}_{\text{mm}}$ are computed on the response scale with the module random-effect BLUP plugged in and uncertainty propagated by 20,000 Monte-Carlo draws from the factored fixed-effects posterior. Fit converged on the first attempt; no fallback (GEE; drop u_{module} ; cluster-robust GLM) was triggered.

Results. The condition main effect is $\hat{\beta}_{\text{cond}} = +4.20$ (SE 0.13, $z = +33.4$, two-sided $p < 10^{-200}$). The condition $\times$ third-party interaction is $\hat{\beta}_{\text{c} \times \text{lib}} = +2.23$ (SE 0.18, $z = +12.3$): the cold-vs-mismatched gap is markedly larger on third-party modules than on stdlib, consistent with the observed per-cell deltas. Random-effect SDs: $\hat{\sigma}_{\text{bug}} = 0.72$ (95% [0.67, 0.77]), $\hat{\sigma}_{\text{module}} = 0.77$ (95% [0.44, 1.35]). All 14/14 cells yield a strictly-positive predicted marginal contrast $\hat{\Delta}_{\text{cell}}$ with 95% MC interval excluding 0 (range +37 pp to +91 pp). Per-cell Wald/permutation/McNemar comparison is in Table 3; cross-phase pooled comparison is in Table 1. The 14-cell qualitative finding of v1.3 (cold > cold_mismatched) thus survives the change of primary inferential procedure.

Comparators. The cluster-robust paired permutation test of v1.3 is retained as the *matched comparator*. For each cell with $n = 30$ paired outcomes (c_i, m_i) we use the same statistic $T = \frac{1}{n} \sum_i (c_i - m_i)$, sample $n_{\text{perm}} = 20,000$ within-pair permutations under the null of label exchangeability, and report the one-sided p . Across $K = 14$ cells a block bootstrap over cells [7] provides a cross-phase 95% percentile CI for the pooled Δ . The legacy McNemar exact one-sided test (Equation (19)) is reported in Table 3 as a literature-comparator only; it inflates significance under within-cell dependence and is not relied on for any v1.4 claim.

Pre-registered acceptance criterion. A v1.4 acceptance criterion was that the 14/14 cells previously significant under the permutation comparator must remain qualitatively significant under the mixed-effects per-cell marginal contrast at $\alpha = 0.025$ (one-sided) — equivalently, the 95% MC interval on $\hat{\Delta}_{\text{cell}}$ must exclude 0. The criterion is satisfied with no discrepancies (`experiments/mixed_effects/discrepancies.md`). The reproduction artefacts are `experiments/mixed_effects/{export_longformat.py, fit.py, comparison_tables.py}`

writing to `long_format.csv`, `results.json`, and `tables.json`. The permutation comparator is unchanged at `experiments/cluster_robust.py` and `cluster_robust_c1.py` writing to `results_cluster_robust.json`.

A.6 Parse-failure policies

For each reviewer call, the output may fail to parse (no valid JSON with `bugs_found` field). We report three policies:

- **Strict (default in Phases 9–11):** unparsed counts as a miss.
- **Lenient:** unparsed dropped from the denominator (per-parsed rate).
- **Mid:** unparsed counts as 0.5 catch.

The Phase-4 finding “paired McNemar significant” is robust to all three policies under Bonferroni.

A.7 Detection-criterion specification

A reviewer’s output “detects” a bug iff the `bugs_found` array contains a string that satisfies at least one of:

- the bug’s reference line number, or its ± 2 neighbours, appear as a number in the string;
- the enclosing function/class name (extracted from the AST) appears in the string AND at least one operator-keyword (from the manifest’s category-keyword list) appears in the string.

This criterion is computed by `analyze_p{N}.py` from frozen detection manifests built by `build_detection.py`. The same criterion is used across all P9–P11 phases.

A.8 Pre-registration discipline

Every phase’s pre-registration document is committed to the repository before any reviewer call, with:

- the frozen hypotheses (H_1 , H_0);
- the frozen primary and secondary metrics;
- the frozen α thresholds (Bonferroni-corrected where applicable);
- the frozen decision rules (what would constitute confirmation / rejection / inconclusive);
- the frozen sample sizes and inclusion criteria;
- the explicit acknowledgement of risks (training-data bias, ablation incoherence, single-family limitation, etc.).

After observing results, we do not iterate prompts, contracts, or sampling. Reframings are applied only to claim language, not to the experimental setup.

Table 3: C1 per-cell three-way comparison referenced from Section 7.8: legacy McNemar exact (one-sided), cluster-robust paired permutation (one-sided, $n_{\text{perm}} = 20,000$, comparator), and the mixed-effects predicted marginal contrast $\hat{\Delta}_{\text{cell}}$ from Section A.5 (primary). All p -values rounded; permutation $p \leq 5 \cdot 10^{-5}$ indicates the minimum achievable under 20,000 permutations. The mixed-effects 95% MC interval is on the response scale (percentage points), with the per-module random-effect BLUP plugged in.

Cell	n	McNemar p	Perm. p	ME $\hat{\Delta}$ (pp)	ME 95% CI (pp)
P9 csv (Codex)	30	$1.2 \cdot 10^{-4}$	$2.5 \cdot 10^{-4}$	+38.6	[+31.1, +46.5]
P9 urllib (Codex)	30	$3.1 \cdot 10^{-5}$	$5.0 \cdot 10^{-5}$	+46.5	[+38.5, +54.5]
P9 jsondec (Codex)	30	$1.2 \cdot 10^{-4}$	$1.5 \cdot 10^{-4}$	+41.4	[+33.6, +49.4]
P9 csv (Opus)	30	$3.9 \cdot 10^{-3}$	$4.4 \cdot 10^{-3}$	+37.1	[+22.5, +53.6]
P9 csv (Gemini 3.1)	30	$3.8 \cdot 10^{-6}$	$5.0 \cdot 10^{-5}$	+53.7	[+35.9, +69.5]
P10 dateutil (Codex)	30	$3.7 \cdot 10^{-9}$	$5.0 \cdot 10^{-5}$	+91.0	[+86.6, +93.4]
P10 dateutil (Opus)	30	$3.7 \cdot 10^{-9}$	$5.0 \cdot 10^{-5}$	+87.4	[+81.0, +90.3]
P10 dateutil (Gemini 3.1)	30	$1.5 \cdot 10^{-8}$	$5.0 \cdot 10^{-5}$	+90.5	[+86.4, +92.5]
P10 parsy (Codex)	30	$1.5 \cdot 10^{-5}$	$1.0 \cdot 10^{-4}$	+65.2	[+51.2, +77.0]
P10 parsy (Opus)	30	$3.1 \cdot 10^{-5}$	$1.0 \cdot 10^{-4}$	+64.9	[+44.4, +80.6]
P10 parsy (Gemini 3.1)	30	$3.8 \cdot 10^{-6}$	$5.0 \cdot 10^{-5}$	+78.2	[+60.3, +88.8]
P10 chardet (Codex)	30	$1.9 \cdot 10^{-9}$	$5.0 \cdot 10^{-5}$	+89.8	[+84.3, +92.9]
P10 chardet (Opus)	30	$3.7 \cdot 10^{-9}$	$5.0 \cdot 10^{-5}$	+86.9	[+78.8, +90.5]
P10 chardet (Gemini 3.1)	30	$3.7 \cdot 10^{-9}$	$5.0 \cdot 10^{-5}$	+90.8	[+86.1, +92.9]

A.9 C1 per-cell three-way comparison (full table)

References

- [1] J. H. Andrews, L. C. Briand, Y. Labiche, and A. S. Namin. Using mutation analysis for assessing and comparing testing coverage criteria. *IEEE Transactions on Software Engineering*, 32(8):608–624, 2006.
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [4] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.
- [5] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning (ICML)*, 2024.
- [6] Aryeh Dvoretzky, Jack Kiefer, and Jacob Wolfowitz. Asymptotic minimax character of the sample distribution function and of the classical multinomial estimator. *The Annals of Mathematical Statistics*, 27(3):642–669, 1956.
- [7] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1994.
- [8] Ronald A. Fisher. *Statistical Methods for Research Workers*. Oliver and Boyd, Edinburgh, 1925.

- [9] Josef Hadar and William R. Russell. Rules for ordering uncertain prospects. *The American Economic Review*, 59(1):25–34, 1969.
- [10] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.
- [11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [12] René Just, Darioush Jalali, and Michael D. Ernst. Defects4j: a database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*, pages 437–440. ACM, 2014.
- [13] Haim Levy. Stochastic dominance and expected utility: Survey and analysis. *Management Science*, 38(4):555–593, 1992.
- [14] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [15] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [16] Inbal Magar and Roy Schwartz. Data contamination: From memorization to exploitation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [17] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- [18] Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall, 2 edition, 1997.
- [19] A. Jefferson Offutt, Ammei Lee, Gregg Rothermel, Roland H. Untch, and Christian Zapf. An experimental determination of sufficient mutant operators. In *ACM Transactions on Software Engineering and Methodology (TOSEM)*, volume 5, pages 99–118. ACM, 1996.
- [20] Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.
- [21] Skipper Seabold and Josef Perktold. statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference (SciPy 2010)*, pages 92–96, 2010.
- [22] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [23] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

- [24] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [25] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012.
- [26] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [27] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. In *arXiv preprint arXiv:2407.01489*, 2024.
- [28] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [29] Kun Zhou, Yutao Zhu, Zhipeng Chen, Wentong Chen, Wayne Xin Zhao, Xu Chen, Yankai Lin, Ji-Rong Wen, and Jiawei Han. Don’t make your LLM an evaluation benchmark cheater. *arXiv preprint arXiv:2311.01964*, 2023.
- [30] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.